8 ELEMENTI FONDAMENTALI PER LA PROGRAMMAZIONE

OBIETTIVI

- Mostrare come applicare la logica nei linguaggi di programmazione.
- Dimostrare le buone pratiche nell'uso di loop e strutture condizionali.
- Discutere gli stili di programmazione più importanti.
- Spiegare come gestire lo **stato** di un programma e i suoi effetti.
- Dimostrare costrutti avanzati che rendono i programmi più facili da gestire.

LOGICA

Nel **Capitolo 2** abbiamo introdotto il **pensiero logico**. In questa sezione vedremo come applicarlo nei linguaggi di programmazione, utilizzando Python per gli esempi.

Le proposizioni logiche in Python

In logica, una **proposizione** è un'affermazione che può essere **vera** o **falsa**. In Python, non esistono proposizioni logiche vere e proprie, ma si usano le **espressioni booleane**, che restituiscono True o False.

Esempi:

python

CopiaModifica

```
x = 10

print(x > 5)   # True

print(x < 3)   # False
```

Operatori logici in Python

Operatore	Parola chiave	Esempio
AND	and	soleggiato and vacanza
OR	or	giocatore_vince or partita_finita
NOT	not	not (punteggio < 50 or punteggio > 80)

Le parentesi **migliorano la leggibilità** e permettono di controllare l'ordine delle operazioni.

COSTRUTTI ALGORITMICI DI BASE

Tutti gli algoritmi, anche quelli complessi, si basano su **tre costrutti fondamentali**:

1⃣ **Sequenza** → Istruzioni eseguite una dopo l'altra.
2⃣ **Condizionale** → Esegue istruzioni solo se una condizione è vera.
3⃣ **Iterazione** → Ripete un'azione finché una condizione è vera.

Esempio di sequenza

python

CopiaModifica

```
nome = input('Come ti chiami? ')

colore = input('Qual è il tuo colore preferito? ')

print('Ti chiami', nome, 'e il tuo colore preferito è', colore)
```

📌 **Attenzione**:

- Una riga di codice solitamente corrisponde a una singola istruzione.
- Se un'istruzione è troppo lunga, si può **spezzare con ** per migliorarne la leggibilità.

CONDIZIONALI

Struttura base di un if

python

CopiaModifica

```
if numero_sedie == 0:

  in_piedi = True
```

💡 **Regole importanti**
✅ Python **usa l'indentazione** per definire i blocchi di codice.
✅ Un'istruzione che apre un nuovo blocco (come if) **termina con :**.

Condizionali complessi con elif

python

CopiaModifica

```
if punteggio >= 80:

  voto = 'A'

elif punteggio >= 70:
```

```
    voto = 'B'
elif punteggio >= 60:
    voto = 'C'
elif punteggio >= 50:
    voto = 'D'
else:
    voto = 'F'
```

📌 **Ordine delle condizioni**: devono essere ordinate in modo logico.
Esempio errato:

python

CopiaModifica

```
if punteggio >= 50:
    voto = 'D'
elif punteggio >= 60:
    voto = 'C'  # Questa riga non verrà mai eseguita
```

💡 **Suggerimento**: ordina sempre le condizioni dal valore più alto al più basso.

ITERAZIONE (LOOP)

Python supporta due tipi di cicli:

1️⃣ **while** → ripete le istruzioni finché una condizione è vera.
2️⃣ **for** → esegue un'azione per ogni elemento in una sequenza.

Esempio di while loop

python

CopiaModifica

```
età = 25
tentativo = 0

while tentativo != età:
    tentativo = int(input('Indovina la mia età: '))
```

print('Hai indovinato!')

Esempio di for loop

python

CopiaModifica

```
parole = ['casa', 'sole', 'luna']

for parola in parole:

   print(len(parola))
```

GESTIONE DELLO STATO

Stato = insieme dei valori delle variabili in un determinato momento.

Esempio:

python

CopiaModifica

```
x = 10

print(x)  # Stampa 10

x = x + 1

print(x)  # Stampa 11
```

📌 **Variabili immutabili vs mutabili**

- **Immutabili**: numeri, stringhe, booleani.
- **Mutabili**: liste, dizionari.

Esempio di problema con liste mutabili:

python

CopiaModifica

```
lista1 = ['A', 'B']

lista2 = lista1

lista1.append('C')

print(lista2)  # ['A', 'B', 'C']
```

👉 **Soluzione**: usa .copy() per creare una copia indipendente:

python

CopiaModifica

```
lista2 = lista1.copy()
```

STILI DI PROGRAMMAZIONE

Python supporta due stili principali:

1️⃣ **Imperativo** → Spiega **come** fare qualcosa (passaggi dettagliati).
2️⃣ **Dichiarativo** → Spiega **cosa** fare, senza specificare i dettagli.

Esempio imperativo:

python

CopiaModifica

```
numeri = [1, 2, 3, 4, 5]
nuovi_numeri = []

for n in numeri:
  if n % 2 == 0:
    nuovi_numeri.append(n)

print(nuovi_numeri)
```

Esempio dichiarativo:

python

CopiaModifica

```
nuovi_numeri = list(filter(lambda x: x % 2 == 0, [1, 2, 3, 4, 5]))
print(nuovi_numeri)
```

📌 **Vantaggi dello stile dichiarativo**
✅ Codice più breve e leggibile.
✅ Meno gestione dello stato.
✅ Facile da testare e mantenere.

ESERCIZI

1⃣ Vero o Falso

- continue termina un loop immediatamente.
- La programmazione dichiarativa è utile per controllare passo per passo il comportamento di un programma.
- In un'istruzione if, solo una condizione può essere vera.

2⃣ Liste mutabili
Modifica il codice in modo che l'aggiornamento di lista1 **non modifichi** anche lista2.

3⃣ Convertire Fahrenheit in Celsius
Scrivi un programma che converta una lista di temperature Fahrenheit in Celsius con la formula:

C=(F−32)×59C = (F - 32) \times \frac{5}{9}C=(F−32)×95

4⃣ Filtrare parole fastidiose
Scrivi un programma che rimuova le parole:

- IN MAIUSCOLO
- Con **più punti esclamativi** (!!!)